

Desarrollador

1. Verificación de Coherencia Estructural con E10 (Trazabilidad de Operaciones) Se ha realizado una auditoría técnica para garantizar que el comportamiento dinámico modelado en los Diagramas de Secuencia (E10) esté soportado por la estructura estática del Diagrama de Clases (E11). Se confirma que el 100% de los métodos críticos han sido mapeados a sus clases correspondientes sin invenciones de lógica:

- En la capa de Control (Controllers):
 - Ctrl_Documento: Se integró el método solicitarCarga(payload) proveniente del CU-02.
 - Ctrl_Postulacion: Se integró el método procesarPostulacion(user_id, vacante_id) proveniente del CU-05.
- En la capa de Servicio (Services):
 - Svc_Validacion: Implementa validarArchivo(payload) y la validación de seguridad crítica inspeccionarMagicNumber().
 - Svc_Expediente: Implementa verificarCompleitud(user_id, vacante_id) para orquestar la regla de negocio principal.
- En la capa de Persistencia (Repositories):
 - Repo_ObjectStore: Implementa almacenar(archivo).
 - Repo_BD: Consolida los métodos transaccionales relacionales: asociarArchivoAspirante(user_id, url), ejecutarCountDocumentos(user_id), consultarObligatorios(vacante_id) y actualizarEstatus(user_id, estatus).

2. Identificación de Clases Críticas Desde la perspectiva de la viabilidad y seguridad del código, se identifican las siguientes clases como los puntos de fallo único (Single Points of Failure) que requerirán mayor cobertura de pruebas unitarias (Unit Testing):

- Svc_Validacion: Es la clase más crítica a nivel de seguridad. Al ser la responsable de procesar el encabezado binario (Magic Number), una falla en sus métodos permitiría la inyección de malware en la infraestructura, evadiendo la validación de extensiones falsificadas.
- Svc_Expediente: Crítica a nivel de reglas de negocio. Esta clase funciona como el Gatekeeper del sistema; si su método de validación de completitud falla por una condición de carrera, el sistema podría registrar contrataciones inválidas.
- Repo_BD: Crítica a nivel de integridad transaccional (ACID). Es la única clase autorizada para ejecutar operaciones de mutación de estado (UPDATE) en la base de datos relacional.

3. Análisis de Acoplamiento (Coupling) El modelo estático presenta un Bajo Acoplamiento general. Esto se logró respetando estrictamente la dirección de las dependencias arquitectónicas.

- Los Controladores (Ctrl_Documento, Ctrl_Postulacion) son completamente agnósticos a la tecnología de base de datos; no dependen de los repositorios, sino únicamente de las interfaces de los Servicios.
- El almacenamiento físico está desacoplado: La persistencia relacional (Repo_BD) y el almacenamiento de archivos masivos (Repo_ObjectStore) son clases independientes. Si en el futuro se migra de un servidor local a AWS S3, solo será necesario modificar Repo_ObjectStore sin afectar al resto del sistema.

4. Análisis de Cohesión (Cohesion) El diagrama exhibe una Alta Cohesión , cumpliendo rigurosamente con el Principio de Responsabilidad Única (SRP).

- Ningún Controlador contiene bloques lógicos de cálculos estadísticos o validaciones de strings/binarios.
- Las Entidades (Usuario, Documento, Expediente) actúan como POJOs (Plain Old Java Objects) o modelos de dominio puros; estructuran la información persistente pero no contienen lógica de base de datos, garantizando que el diseño sea escalable y fácil de mantener en los próximos sprints de desarrollo.